4.3 浏览器工具与 Canvas 画布

> **生成模型**：GPT-5.4 (openai/gpt-5.4)
> **Token 消耗**：输入 ~50k tokens，输出 ~3.8k tokens（本节）

到这一节，OpenClaw 的味道就更明显了。前面我们已经看到它可以聊天、能跑 Bash。再往前一步，它甚至可以“看网页”“点页面”“抓数据”“给你推一个可交互界面”。这听起来像两个不同能力，其实都指向同一个方向：让 AI 不只在文本框里工作，而是开始接触真实界面。

这一节分成两部分讲：先讲**浏览器工具**，再讲 **Canvas**。前者可以理解成“AI 去操作现成网页”，后者可以理解成“AI 自己给你做一个界面并推过来”。

4.3.1 浏览器工具：不是只会读网页，而是真的会操作页面

很多聊天模型也能“总结网页”，但大多依赖你把网页内容先贴给它。OpenClaw 的浏览器工具更进一步：它可以自己打开网页、等待页面加载、查看页面内容、做点击、截图、提取数据。

如果把 Bash 比作“操作系统的手”，那浏览器工具就像“屏幕里的手”。

最常见的用途包括：

- 打开某个文档网站，提取重点
- 登录后查看某个页面信息
- 截图网页，确认布局或内容
- 在页面上点击按钮、填写表单
- 从列表页批量提取数据

这类能力特别适合“网页本来就有信息，但你懒得自己一页页看”的情况。

4.3.2 对非技术用户来说，它到底在干什么

你可以把浏览器工具理解成：OpenClaw 背后连着一个受控浏览器。你给它任务，它会像人一样走几个步骤：

1. 打开页面

2. 等内容出来
3. 找到你关心的区域
4. 必要时点击或滚动
5. 把看到的内容整理回来

这跟单纯“抓 HTML”不一样。因为现代网站很多内容是 JavaScript 动态渲染的，只看初始源码可能啥也没有。浏览器工具的价值就在于，它看到的是“用户真正看到的页面”。

4.3.3 适合学生的几个高频用法

用法一：读文档

这是最轻松也最实用的一类。

例如：

```
用户：请打开 React 官方文档的 `useEffect` 页面，用中文总结“它什么时候该用，什么时候不该用”。
AI：我会先打开页面，提取关键段落，再用更容易懂的话总结。
```

这种任务的好处是，你不用自己复制粘贴半天，也不用担心模型瞎编文档内容。

用法二：网页数据提取

比如课程通知页、比赛榜单、开源项目 release 页面、实验室公告页，都很适合。

```
用户：请打开这个比赛页面，帮我提取报名截止时间、参赛要求和奖项设置。
AI：我会读取页面内容，并按这三项整理返回。
```

用法三：截图和确认界面

如果你在做网页作业或者前端项目，截图功能很实用。

```
用户：请打开我本地的页面，截一张首页图，并告诉我导航栏有没有挤压。
AI：我会先打开页面截图，再根据页面布局给你一个简短判断。
```

这已经不是“听你描述 bug”，而是它真的去看界面了。

4.3.4 一段浏览器工具的实战对话

```
用户：请打开 OpenClaw 的官方文档首页，找出“快速开始”相关入口，再用 5 条告诉我第一次上手最该看的内容。
AI（调用浏览器工具）：我会先浏览首页和相关入口，再把新手最值得先看的部分提炼出来。
```

这背后的工具调用逻辑，通常会是：

1. 打开目标网址。
2. 观察页面导航和主要卡片。
3. 必要时进入“快速开始”页。
4. 提取标题、说明、示例命令。
5. 汇总成 5 条建议。

对用户来说，最重要的感受是：你不用自己先把网页内容喂给它，它可以自己去读。

4.3.5 浏览器工具的边界：它不是万能爬虫

虽然浏览器工具很强，但你也别把它想成“整个互联网随便拿捏”。从使用者角度，至少要知道下面几个限制。

第一，有些页面需要登录，而且登录流程可能有验证码、多因素认证或者设备限制，这时工具未必能顺利通过。

第二，有些网站内容非常重，加载慢、交互复杂、前端框架很花，AI 操作成功率会受影响。

第三，网页能看到，不代表就适合批量抓。数据提取要尊重网站规则，也要注意别把自己用成违规爬虫。

第四，如果你的目标很模糊，比如“看看这个网站有啥”，那它容易给出一堆泛泛总结；如果你说“提取这三个字段”，效果通常就好很多。

所以浏览器工具的最佳用法，往往不是“随便逛”，而是“带着明确目标去看”。

4.3.6 Canvas 是什么：让 AI 把结果直接做成界面

前面的浏览器工具，操作的是别人的网页。Canvas 则完全不同，它更像 OpenClaw 自带的一块画布，AI 可以把 HTML 或 React 风格的界面推到这块画布上，让你直接看、直接点、直接交互。

如果你第一次接触这个概念，可以这样想：

- 浏览器工具：AI 去外面的网页里干活。
- Canvas：AI 在自己的地盘上给你搭一个小界面。

这就不只是“回你一段 Markdown”了。它可以给你一个可视化卡片、一个表格面板、一个交互式页面，甚至一个小工具原型。

4.3.7 A2UI：把“AI 说话”升级成“AI 直接出界面”

A2UI 是 Agent-to-UI 的缩写，可以翻成“从 Agent 到界面”。名字听上去有点硬，其实核心意思很朴素：**AI 不只是回文字，它还能直接生成用户界面。**

为什么这个概念重要？因为很多结果，本来就不适合只用纯文字表达。

比如：

- 课程安排表
- 实验数据仪表板
- 旅行计划卡片
- 倒计时和待办面板
- 一个可以点按钮的小 Demo

这些东西如果硬塞在聊天框里，要么很丑，要么不好用。Canvas/A2UI 的价值，就是让 AI 产出的东西从“回答”变成“界面产物”。

你甚至可以把它想成一种很轻量的“AI 前端生成能力”：你描述要什么界面，OpenClaw 就把页面推给你看。

4.3.8 怎么把 HTML 或 React 风格界面推到 Canvas

这里很多人第一次会误会，以为 Canvas 只能显示一张截图。不是。它本质上更像一个被 OpenClaw 控制的 Web 容器，所以最自然的内容就是网页界面。

最常见的两种内容来源是：

- **HTML 页面**：AI 直接生成一个完整 HTML 文件，里面带上样式和交互脚本，然后推到 Canvas。
- **React 风格页面**：AI 按组件化思路组织界面，再打包成最终能渲染的页面，推到 Canvas 展示。

对普通读者来说，不用纠结底层细节，你只要记住：Canvas 吃的是“前端界面产物”。你让它做一个页面，它就可以把这个页面投到画布上。

一个很实用的说法是：

```
用户：请做一个 HTML 版课程表页面，推到 Canvas，手机宽度优先。
AI（调用 Canvas 工具）：我会生成页面并推送到 Canvas，重点保证手机上易读。
```

如果你更习惯组件化表达，也可以这样说：

```
用户：请按 React 组件思路做一个番茄钟界面，包含计时器、开始按钮、暂停按钮和今日统计，再推到 Canvas。
AI（调用 Canvas 工具）：我会按组件结构组织这个界面，并把最终页面推送到 Canvas。
```

这里的关键词不是 HTML 还是 React，而是你把界面需求说清楚了：显示什么、怎么布局、重点在哪块屏幕上看。

4.3.9 Canvas surfaces：它会显示到哪里

你可以把 surface 理解成“画布显示面”。OpenClaw 的 Canvas 不是只属于桌面浏览器，它可以出现在不同设备的界面上。

最常见的 surface 包括：

- macOS 上的窗口或嵌入视图
- iPhone / iPad 上的 App 内画布区域
- Android 手机上的画布区域

这件事很关键，因为它决定了 Canvas 不是一个只适合开发者演示的小玩具。你完全可以把它当成跨设备的展示面板：在电脑上做出来，在手机上看；或者让 AI 直接生成一个更适合移动端阅读的小界面。

所以你在下指令时，最好顺手加上设备感知：

- “手机上优先，单列布局”
- “桌面端可以两栏”
- “字号大一点，适合触屏点击”

这种描述对 Canvas 特别有效，因为它本来就是面向界面的，不只是面向文字的。

4.3.10 Canvas 最适合哪些场景

场景一：把信息整理成可视化面板

例如：

```
用户：把我这周的课程安排做成一个手机上好读的卡片式页面，推到 Canvas。
AI：我会生成一个适合手机查看的页面，并把课程按天整理出来。
```

这时候 AI 不是回一张表，而是直接给你一个更像产品原型的东西。

场景二：快速做前端原型

对学生做课程设计特别有用。你不用先开完整工程，也能快速看一个页面概念。

```
用户：给我做一个“图书借阅系统首页”的简洁原型，包含搜索框、推荐书籍和借阅记录入口。
AI：我会先生成一个静态界面原型，方便你确认布局和信息结构。
```

场景三：做交互式小工具

比如计算器、打卡板、排班表、待办板，这些都很适合 Canvas。

因为它的重点不是复杂后端，而是“把一个能用的交互界面先立起来”。

4.3.11 一段 Canvas 的实战对话

```
用户：请做一个宿舍值日排班页面，风格干净一点，手机上能看，推到 Canvas。
AI（调用 Canvas 工具）：我会生成一个适合手机浏览的排班页面，包含本周值日安排和轮换规则说明。
```

如果再进一步，你还可以继续说：

```
用户：把“今天值日的人”做成高亮卡片，再加一个“下周轮换预览”。
AI（继续调用 Canvas 工具）：我会更新画布，把当前值日和下周安排区分展示。
```

这类体验很像你在和一个会写前端的同学配合，而且反馈速度非常快。

4.3.12 浏览器工具和 Canvas 的差别，一句话讲透

很多读者第一次会把这两个混在一起。你只要记住下面这张对照表就够了。

维度	浏览器工具	Canvas
干什么	操作已有网页	展示 AI 生成的界面

维度	浏览器工具	Canvas
主要用途	阅读、提取、点击、截图	原型、面板、交互式展示
内容来源	外部网站或本地页面	AI 生成的 HTML / UI
用户感觉	AI 在“帮你逛网页”	AI 在“直接给你做界面”

说得再口语一点：

- 浏览器工具像一个会用网页的研究助理。
- Canvas 像一个能把结果直接摆到你面前的界面助手。

4.3.13 第四章最让人兴奋的一点：AI 开始碰真实世界了

到这里，你应该能明显感觉到 OpenClaw 跟普通聊天模型的分水岭在哪里。

普通聊天模型擅长“描述世界”。

OpenClaw 开始能“接触世界”：

- 用 Bash 碰操作系统
- 用浏览器碰网页
- 用 Canvas 碰界面

这件事为什么重要？因为真实任务往往不是“回答一个概念题”，而是要查、要看、要操作、要生成可交付的结果。也正因为这样，OpenClaw 才更像一个实用型助手，而不只是一个聊天窗口。

本节小结

1. 浏览器工具让 OpenClaw 可以像人一样打开网页、读取内容、截图、点击和提取数据，而不是只靠你手动贴网页文本。
2. 它最适合的场景是读文档、抓取页面关键信息、检查网页界面，以及完成轻量网页交互。
3. Canvas 和浏览器工具不同：前者是 AI 自己生成并推送界面，后者是 AI 去操作现成网页。
4. A2UI 可以简单理解成“AI 直接出界面”，它让很多原本只适合文字描述的结果，变成更直观的可视化产物。
5. 当 Bash、浏览器和 Canvas 放在一起看时，你会发现 OpenClaw 真正厉害的地方不是会聊，而是开始能替你碰终端、碰网页、碰界面。